

原创 C++ 算法基础模板合集

Basic algorithm templates in C++17

非官方资料；不包含 AcWing 课程题面或整套题库答案。

仅作参考复习与开卷参考使用；考试请遵守考场规则。

Sections: 87

目录

- 01. 00. Common Header
- 02. 01. Quick Sort
- 03. 02. Merge Sort
- 04. 03. Inversion Count
- 05. 04. Integer Binary Search
- 06. 05. Floating Binary Search
- 07. 06. High Precision Add
- 08. 07. High Precision Subtract
- 09. 08. High Precision Multiply By Int
- 10. 09. High Precision Divide By Int
- 11. 10. Prefix Sum 1D
- 12. 11. Prefix Sum 2D
- 13. 12. Difference 1D
- 14. 13. Difference 2D
- 15. 14. Two Pointers: Longest Distinct Subarray
- 16. 15. Bit Operations
- 17. 16. Coordinate Compression
- 18. 17. Merge Intervals
- 19. 18. Static Singly Linked List
- 20. 19. Static Doubly Linked List
- 21. 20. Array Stack
- 22. 21. Array Queue
- 23. 22. Monotonic Stack: Nearest Smaller Left
- 24. 23. Monotonic Queue: Sliding Window Min Max
- 25. 24. KMP
- 26. 25. Trie Lowercase
- 27. 26. Disjoint Set Union
- 28. 27. Weighted DSU: Distance To Root
- 29. 28. Manual Min Heap
- 30. 29. Hash Table: Separate Chaining
- 31. 30. Hash Table: Open Addressing
- 32. 31. String Hash
- 33. 32. Fenwick Tree
- 34. 33. Segment Tree: Range Add Range Sum
- 35. 34. Sparse Table RMQ Max
- 36. 35. Adjacency List Graph
- 37. 36. DFS Permutation
- 38. 37. DFS N Queens
- 39. 38. BFS Grid Shortest Path
- 40. 39. Topological Sort
- 41. 40. Dijkstra Dense
- 42. 41. Dijkstra Heap
- 43. 42. Bellman Ford
- 44. 43. SPFA Shortest Path
- 45. 44. SPFA Negative Cycle
- 46. 45. Floyd
- 47. 46. Prim Dense MST
- 48. 47. Kruskal MST
- 49. 48. Bipartite Check By Coloring
- 50. 49. Hungarian Algorithm
- 51. 50. Trial Division Prime
- 52. 51. Linear Sieve
- 53. 52. Divisors
- 54. 53. Divisor Count And Sum Mod
- 55. 54. Fast Power Mod
- 56. 55. Extended GCD And Inverse
- 57. 56. Linear Congruence
- 58. 57. Extended Chinese Remainder
- 59. 58. Gaussian Elimination Double
- 60. 59. Gaussian Elimination XOR
- 61. 60. Combinations Pascal Mod
- 62. 61. Combinations Factorial Mod Prime
- 63. 62. Lucas Theorem
- 64. 63. Catalan Number Mod Prime
- 65. 64. Nim And SG
- 66. 65. 0/1 Knapsack
- 67. 66. Complete Knapsack
- 68. 67. Multiple Knapsack Binary Optimization
- 69. 68. Group Knapsack
- 70. 69. Longest Increasing Subsequence
- 71. 70. Longest Common Subsequence
- 72. 71. Edit Distance
- 73. 72. Interval DP: Stone Merge
- 74. 73. Integer Partition Count
- 75. 74. Digit DP Skeleton

```
76. 75. State Compression DP: Hamilton Path
77. 76. State Compression DP: Mondrian Dream
78. 77. Tree DP: No Boss Party
79. 78. Memoized Search: Skiing
80. 79. Greedy: Interval Selection
81. 80. Greedy: Interval Grouping
82. 81. Greedy: Interval Cover
83. 82. Greedy: Huffman Merge
84. 83. Greedy: Median Absolute Inequality
85. 84. Greedy: Sorting Inequality
86. 85. Fast IO Scanner
87. 86. Output Helpers
```

00. Common Header

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<long long, long long>;

const int INF = 0x3f3f3f3f;
const long long LINF = 4e18;
const int MOD = 1000000007;

int dx4[4] = {-1, 0, 1, 0};
int dy4[4] = {0, 1, 0, -1};
int dx8[8] = {-1,-1,-1,0,0,1,1,1};
int dy8[8] = {-1,0,1,-1,1,-1,0,1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    return 0;
}
```

01. Quick Sort

```
const int N = 100010;
int q[N];

void quick_sort(int q[], int l, int r) {
    if (l >= r) return;
    int i = l - 1, j = r + 1, x = q[(l + r) >> 1];
    while (i < j) {
        do i++; while (q[i] < x);
        do j--; while (q[j] > x);
        if (i < j) swap(q[i], q[j]);
    }
    quick_sort(q, l, j);
    quick_sort(q, j + 1, r);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    for (int i = 0; i < n; i++) cin >> q[i];
    quick_sort(q, 0, n - 1);
    for (int i = 0; i < n; i++) cout << q[i] << ' ';
    return 0;
}
```

02. Merge Sort

```
const int N = 100010;
int q[N], tmp[N];

void merge_sort(int q[], int l, int r) {
    if (l >= r) return;
    int mid = (l + r) >> 1;
    merge_sort(q, l, mid);
    merge_sort(q, mid + 1, r);
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (q[i] <= q[j]) tmp[k++] = q[i++];
        else tmp[k++] = q[j++];
    }
    while (i <= mid) tmp[k++] = q[i++];
    while (j <= r) tmp[k++] = q[j++];
    for (i = l, j = 0; i <= r; i++, j++) q[i] = tmp[j];
}
```

03. Inversion Count

```
const int N = 100010;
int q[N], tmp[N];

long long merge_sort(int l, int r) {
    if (l >= r) return 0;
    int mid = (l + r) >> 1;
    long long res = merge_sort(l, mid) + merge_sort(mid + 1, r);
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (q[i] <= q[j]) tmp[k++] = q[i++];
        else {
            tmp[k++] = q[j++];
            res += mid - i + 1;
        }
    }
    while (i <= mid) tmp[k++] = q[i++];
    while (j <= r) tmp[k++] = q[j++];
    for (i = l, j = 0; i <= r; i++, j++) q[i] = tmp[j];
    return res;
}
```

04. Integer Binary Search

```
int lower_bound_template(vector<int>& a, int x) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (a[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return a.empty() || a[l] < x ? -1 : l;
}

int upper_bound_template(vector<int>& a, int x) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int mid = (l + r + 1) >> 1;
        if (a[mid] <= x) l = mid;
        else r = mid - 1;
    }
    return a.empty() || a[l] > x ? -1 : l;
}
```

05. Floating Binary Search

```
bool check(double x) {
    return x * x >= 2.0;
}

double bsearch_double(double l, double r) {
    const double eps = 1e-8;
    while (r - l > eps) {
        double mid = (l + r) / 2;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return l;
}
```

06. High Precision Add

```
vector<int> add(vector<int>& A, vector<int>& B) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < (int)A.size() || i < (int)B.size(); i++) {
        if (i < (int)A.size()) t += A[i];
        if (i < (int)B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    if (t) C.push_back(t);
    return C;
}

vector<int> read_big(const string& s) {
    vector<int> A;
    for (int i = (int)s.size() - 1; i >= 0; i--) A.push_back(s[i] - '0');
    return A;
}

void print_big(const vector<int>& A) {
    for (int i = (int)A.size() - 1; i >= 0; i--) cout << A[i];
}
```

```
}
```

07. High Precision Subtract

```
bool cmp(vector<int>& A, vector<int>& B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = (int)A.size() - 1; i >= 0; i--)
        if (A[i] != B[i]) return A[i] > B[i];
    return true;
}
```

```
vector<int> sub(vector<int>& A, vector<int>& B) {
    vector<int> C;
    for (int i = 0, t = 0; i < (int)A.size(); i++) {
        t = A[i] - t;
        if (i < (int)B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        t = t < 0;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

08. High Precision Multiply By Int

```
vector<int> mul(vector<int>& A, int b) {
    vector<int> C;
    long long t = 0;
    for (int i = 0; i < (int)A.size() || t; i++) {
        if (i < (int)A.size()) t += 1LL * A[i] * b;
        C.push_back(t % 10);
        t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

09. High Precision Divide By Int

```
vector<int> div(vector<int>& A, int b, int& r) {
    vector<int> C;
    r = 0;
    for (int i = (int)A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

10. Prefix Sum 1D

```
const int N = 100010;
int a[N], s[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i], s[i] = s[i - 1] + a[i];
    while (m--) {
        int l, r; cin >> l >> r;
        cout << s[r] - s[l - 1] << '\n';
    }
    return 0;
}
```

11. Prefix Sum 2D

```
const int N = 1010;
int a[N][N], s[N][N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q; cin >> n >> m >> q;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j];
        }
    }
    while (q--) {
```

```
        int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        cout << s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1]
            + s[x1 - 1][y1 - 1] << '\n';
    }
    return 0;
}
```

12. Difference 1D

```
const int N = 100010;
int a[N], b[N];

void insert(int l, int r, int c) {
    b[l] += c;
    b[r + 1] -= c;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i], insert(i, i, a[i]);
    while (m--) {
        int l, r, c; cin >> l >> r >> c;
        insert(l, r, c);
    }
    for (int i = 1; i <= n; i++) b[i] += b[i - 1], cout << b[i] << ' ';
    return 0;
}
```

13. Difference 2D

```
const int N = 1010;
int a[N][N], b[N][N];

void insert(int x1, int y1, int x2, int y2, int c) {
    b[x1][y1] += c;
    b[x2 + 1][y1] -= c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y2 + 1] += c;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q; cin >> n >> m >> q;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) cin >> a[i][j], insert(i, j, i, j, a[i][j]);
    while (q--) {
        int x1, y1, x2, y2, c; cin >> x1 >> y1 >> x2 >> y2 >> c;
        insert(x1, y1, x2, y2, c);
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i - 1][j - 1];
            cout << b[i][j] << ' ';
        }
        cout << '\n';
    }
    return 0;
}
```

14. Two Pointers: Longest Distinct Subarray

```
const int N = 100010;
int a[N], cnt[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    int res = 0;
    for (int i = 0, j = 0; i < n; i++) {
        cnt[a[i]]++;
        while (cnt[a[i]] > 1) cnt[a[j++]]--;
        res = max(res, i - j + 1);
    }
    cout << res << '\n';
    return 0;
}
```

15. Bit Operations

```
int lowbit(int x) {
    return x & -x;
}

int count_bits(int x) {
    int res = 0;
    while (x) x -= lowbit(x), res++;
    return res;
}

bool kth_bit_is_one(long long x, int k) {
    return (x >> k) & 1LL;
}

long long set_kth_bit(long long x, int k) {
    return x | (1LL << k);
}

long long clear_kth_bit(long long x, int k) {
    return x & ~(1LL << k);
}
```

16. Coordinate Compression

```
vector<int> alls;
vector<pair<int, int>> addv, query;

int find_id(int x) {
    int l = 0, r = (int)alls.size() - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1;
}

void compress_prepare() {
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin(), alls.end()), alls.end());
}
```

17. Merge Intervals

```
vector<pair<int, int>> merge_intervals(vector<pair<int, int>> segs) {
    vector<pair<int, int>> res;
    sort(segs.begin(), segs.end());
    int st = -2e9, ed = -2e9;
    for (auto [l, r] : segs) {
        if (ed < l) {
            if (st != -2e9) res.push_back({st, ed});
            st = l, ed = r;
        } else ed = max(ed, r);
    }
    if (st != -2e9) res.push_back({st, ed});
    return res;
}
```

18. Static Singly Linked List

```
const int N = 100010;
int head, e[N], ne[N], idx;

void init() {
    head = -1;
    idx = 0;
}

void add_to_head(int x) {
    e[idx] = x;
    ne[idx] = head;
    head = idx++;
}

void add_after(int k, int x) {
    e[idx] = x;
    ne[idx] = ne[k];
    ne[k] = idx++;
}

void remove_after(int k) {
    ne[k] = ne[ne[k]];
}
```

```
}

void remove_head() {
    head = ne[head];
}
```

19. Static Doubly Linked List

```
const int N = 100010;
int e[N], l[N], r[N], idx;

void init() {
    r[0] = 1;
    l[1] = 0;
    idx = 2;
}

void add_right(int k, int x) {
    e[idx] = x;
    l[idx] = k;
    r[idx] = r[k];
    l[r[k]] = idx;
    r[k] = idx++;
}

void add_left(int k, int x) {
    add_right(l[k], x);
}

void remove_node(int k) {
    r[l[k]] = r[k];
    l[r[k]] = l[k];
}
```

20. Array Stack

```
const int N = 100010;
int stk[N], tt;

void push(int x) { stk[++tt] = x; }
void pop() { tt--; }
bool empty() { return tt == 0; }
int top() { return stk[tt]; }
```

21. Array Queue

```
const int N = 100010;
int q[N], hh, tt = -1;

void push(int x) { q[++tt] = x; }
void pop() { hh++; }
bool empty() { return hh > tt; }
int front() { return q[hh]; }
```

22. Monotonic Stack: Nearest Smaller Left

```
const int N = 100010;
int stk[N], tt;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    while (n--) {
        int x; cin >> x;
        while (tt && stk[tt] >= x) tt--;
        if (tt) cout << stk[tt] << ' ';
        else cout << -1 << ' ';
        stk[++tt] = x;
    }
    return 0;
}
```

23. Monotonic Queue: Sliding Window Min Max

```
const int N = 1000010;
int a[N], q[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k; cin >> n >> k;
    for (int i = 0; i < n; i++) cin >> a[i];
    int hh = 0, tt = -1;
    for (int i = 0; i < n; i++) {
```

```
        if (hh <= tt && q[hh] < i - k + 1) hh++;
        while (hh <= tt && a[q[tt]] >= a[i]) tt--;
        q[++tt] = i;
        if (i >= k - 1) cout << a[q[hh]] << ' ';
    }
    cout << '\n';
    hh = 0, tt = -1;
    for (int i = 0; i < n; i++) {
        if (hh <= tt && q[hh] < i - k + 1) hh++;
        while (hh <= tt && a[q[tt]] <= a[i]) tt--;
        q[++tt] = i;
        if (i >= k - 1) cout << a[q[hh]] << ' ';
    }
    return 0;
}
```

24. KMP

```
const int N = 100010, M = 1000010;
char p[N], s[M];
int ne[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> (p + 1) >> m >> (s + 1);
    for (int i = 2, j = 0; i <= n; i++) {
        while (j && p[i] != p[j + 1]) j = ne[j];
        if (p[i] == p[j + 1]) j++;
        ne[i] = j;
    }
    for (int i = 1, j = 0; i <= m; i++) {
        while (j && s[i] != p[j + 1]) j = ne[j];
        if (s[i] == p[j + 1]) j++;
        if (j == n) {
            cout << i - n << ' ';
            j = ne[j];
        }
    }
    return 0;
}
```

25. Trie Lowercase

```
const int N = 100010;
int son[N][26], cnt[N], idx;

void insert(const string& str) {
    int p = 0;
    for (char ch : str) {
        int u = ch - 'a';
        if (!son[p][u]) son[p][u] = ++idx;
        p = son[p][u];
    }
    cnt[p]++;
}

int query(const string& str) {
    int p = 0;
    for (char ch : str) {
        int u = ch - 'a';
        if (!son[p][u]) return 0;
        p = son[p][u];
    }
    return cnt[p];
}
```

26. Disjoint Set Union

```
const int N = 100010;
int p[N], sz[N];

int find(int x) {
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

void init(int n) {
    for (int i = 1; i <= n; i++) p[i] = i, sz[i] = 1;
}

void unite(int a, int b) {
    a = find(a), b = find(b);
```



```
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);
    p[b] = a;
    sz[a] += sz[b];
}
```

27. Weighted DSU: Distance To Root

```
const int N = 100010;
int p[N], d[N], sz[N];

int find(int x) {
    if (p[x] != x) {
        int root = find(p[x]);
        d[x] += d[p[x]];
        p[x] = root;
    }
    return p[x];
}

void init(int n) {
    for (int i = 1; i <= n; i++) p[i] = i, d[i] = 0, sz[i] = 1;
}

void merge_chain(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa != pb) {
        p[pa] = pb;
        d[pa] = sz[pb];
        sz[pb] += sz[pa];
    }
}

int distance_between(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa != pb) return -1;
    return max(0, abs(d[a] - d[b]) - 1);
}
```

28. Manual Min Heap

```
const int N = 100010;
int h[N], ph[N], hp[N], sz;

void heap_swap(int a, int b) {
    swap(ph[hp[a]], ph[hp[b]]);
    swap(hp[a], hp[b]);
    swap(h[a], h[b]);
}

void down(int u) {
    int t = u;
    if (u * 2 <= sz && h[u * 2] < h[t]) t = u * 2;
    if (u * 2 + 1 <= sz && h[u * 2 + 1] < h[t]) t = u * 2 + 1;
    if (u != t) heap_swap(u, t), down(t);
}

void up(int u) {
    while (u / 2 && h[u / 2] > h[u]) {
        heap_swap(u / 2, u);
        u /= 2;
    }
}
```

29. Hash Table: Separate Chaining

```
const int N = 100003;
int h[N], e[N], ne[N], idx;

void init() {
    memset(h, -1, sizeof h);
    idx = 0;
}

void insert(int x) {
    int k = (x % N + N) % N;
    e[idx] = x;
    ne[idx] = h[k];
    h[k] = idx++;
}

bool find_hash(int x) {
    int k = (x % N + N) % N;
    for (int i = h[k]; i != -1; i = ne[i])
```

```
        if (e[i] == x) return true;
    return false;
}
```

30. Hash Table: Open Addressing

```
const int N = 200003, nullv = 0x3f3f3f3f;
int h[N];
```

```
int find_slot(int x) {
    int k = (x % N + N) % N;
    while (h[k] != nullv && h[k] != x) {
        k++;
        if (k == N) k = 0;
    }
    return k;
}
```

```
void init() {
    memset(h, 0x3f, sizeof h);
}
```

31. String Hash

```
using ull = unsigned long long;
const int N = 100010, P = 131;
ull h[N], p[N];
```

```
void build_hash(const string& s) {
    p[0] = 1;
    h[0] = 0;
    for (int i = 1; i <= (int)s.size(); i++) {
        p[i] = p[i - 1] * P;
        h[i] = h[i - 1] * P + s[i - 1];
    }
}
```

```
ull get_hash(int l, int r) {
    return h[r] - h[l - 1] * p[r - l + 1];
}
```

32. Fenwick Tree

```
const int N = 100010;
int n, tr[N];
```

```
int lowbit(int x) { return x & -x; }
```

```
void add(int x, int c) {
    for (int i = x; i <= n; i += lowbit(i)) tr[i] += c;
}
```

```
int sum(int x) {
    int res = 0;
    for (int i = x; i; i -= lowbit(i)) res += tr[i];
    return res;
}
```

```
int range_sum(int l, int r) {
    return sum(r) - sum(l - 1);
}
```

33. Segment Tree: Range Add Range Sum

```
const int N = 100010;
long long w[N];
struct Node {
    int l, r;
    long long sum, add;
} tr[N * 4];
```

```
void pushup(int u) {
    tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
}
```

```
void pushdown(int u) {
    auto& root = tr[u];
    auto& left = tr[u << 1];
    auto& right = tr[u << 1 | 1];
    if (root.add) {
        left.add += root.add;
        left.sum += (long long)(left.r - left.l + 1) * root.add;
        right.add += root.add;
        right.sum += (long long)(right.r - right.l + 1) * root.add;
        root.add = 0;
    }
}
```

```
    }  
}  
  
void build(int u, int l, int r) {  
    tr[u] = {l, r, 0, 0};  
    if (l == r) tr[u].sum = w[l];  
    else {  
        int mid = (l + r) >> 1;  
        build(u << 1, l, mid);  
        build(u << 1 | 1, mid + 1, r);  
        pushup(u);  
    }  
}  
  
void modify(int u, int l, int r, long long d) {  
    if (tr[u].l >= l && tr[u].r <= r) {  
        tr[u].sum += (long long)(tr[u].r - tr[u].l + 1) * d;  
        tr[u].add += d;  
    } else {  
        pushdown(u);  
        int mid = (tr[u].l + tr[u].r) >> 1;  
        if (l <= mid) modify(u << 1, l, r, d);  
        if (r > mid) modify(u << 1 | 1, l, r, d);  
        pushup(u);  
    }  
}  
  
long long query(int u, int l, int r) {  
    if (tr[u].l >= l && tr[u].r <= r) return tr[u].sum;  
    pushdown(u);  
    int mid = (tr[u].l + tr[u].r) >> 1;  
    long long res = 0;  
    if (l <= mid) res += query(u << 1, l, r);  
    if (r > mid) res += query(u << 1 | 1, l, r);  
    return res;  
}
```

34. Sparse Table RMQ Max

```
const int N = 100010, K = 18;  
int f[N][K + 1], lg[N];  
  
void build(vector<int>& a, int n) {  
    lg[1] = 0;  
    for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;  
    for (int i = 1; i <= n; i++) f[i][0] = a[i];  
    for (int j = 1; j <= K; j++)  
        for (int i = 1; i + (1 << j) - 1 <= n; i++)  
            f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);  
}  
  
int query(int l, int r) {  
    int k = lg[r - l + 1];  
    return max(f[l][k], f[r - (1 << k) + 1][k]);  
}
```

35. Adjacency List Graph

```
const int N = 100010, M = 200010;  
int h[N], e[M], ne[M], w[M], idx;  
  
void init() {  
    memset(h, -1, sizeof h);  
    idx = 0;  
}  
  
void add_edge(int a, int b, int c = 1) {  
    e[idx] = b;  
    w[idx] = c;  
    ne[idx] = h[a];  
    h[a] = idx++;  
}
```

36. DFS Permutation

```
const int N = 10;  
int n, path[N];  
bool st[N];  
  
void dfs(int u) {  
    if (u == n) {  
        for (int i = 0; i < n; i++) cout << path[i] << ' ';  
        cout << '\n';  
        return;  
    }
```

```
    }
    for (int i = 1; i <= n; i++) {
        if (!st[i]) {
            path[u] = i;
            st[i] = true;
            dfs(u + 1);
            st[i] = false;
        }
    }
}
```

37. DFS N Queens

```
const int N = 20;
int n;
char g[N][N];
bool col[N], dg[N], udg[N];

void dfs(int u) {
    if (u == n) {
        for (int i = 0; i < n; i++) cout << g[i] << '\n';
        cout << '\n';
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!col[i] && !dg[u + i] && !udg[n - u + i]) {
            g[u][i] = 'Q';
            col[i] = dg[u + i] = udg[n - u + i] = true;
            dfs(u + 1);
            col[i] = dg[u + i] = udg[n - u + i] = false;
            g[u][i] = '.';
        }
    }
}
```

38. BFS Grid Shortest Path

```
const int N = 110;
int n, m, g[N][N], d[N][N];
pair<int, int> q[N * N];
int dx[4] = {-1, 0, 1, 0};
int dy[4] = {0, 1, 0, -1};

int bfs() {
    memset(d, -1, sizeof d);
    int hh = 0, tt = 0;
    q[0] = {0, 0};
    d[0][0] = 0;
    while (hh <= tt) {
        auto [x, y] = q[hh++];
        for (int i = 0; i < 4; i++) {
            int a = x + dx[i], b = y + dy[i];
            if (a >= 0 && a < n && b >= 0 && b < m && g[a][b] == 0 && d[a][b] == -1) {
                d[a][b] = d[x][y] + 1;
                q[++tt] = {a, b};
            }
        }
    }
    return d[n - 1][m - 1];
}
```

39. Topological Sort

```
const int N = 100010;
int n, m, h[N], e[N], ne[N], idx, d[N], q[N];

bool topsort() {
    int hh = 0, tt = -1;
    for (int i = 1; i <= n; i++)
        if (!d[i]) q[++tt] = i;
    while (hh <= tt) {
        int t = q[hh++];
        for (int i = h[t]; i != -1; i = ne[i]) {
            int j = e[i];
            if (--d[j] == 0) q[++tt] = j;
        }
    }
    return tt == n - 1;
}
```

40. Dijkstra Dense

```
const int N = 510;
int n, m, g[N][N], distv[N];
bool st[N];
```

```
int dijkstra() {
    memset(distv, 0x3f, sizeof distv);
    distv[1] = 0;
    for (int i = 0; i < n; i++) {
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || distv[j] < distv[t])) t = j;
        st[t] = true;
        for (int j = 1; j <= n; j++)
            distv[j] = min(distv[j], distv[t] + g[t][j]);
    }
    return distv[n] == 0x3f3f3f3f ? -1 : distv[n];
}
```

41. Dijkstra Heap

```
const int N = 100010, M = 200010;
int n, m, h[N], e[M], w[M], ne[M], idx, distv[N];
bool st[N];

void add(int a, int b, int c) {
    e[idx] = b; w[idx] = c; ne[idx] = h[a]; h[a] = idx++;
}

int dijkstra() {
    memset(distv, 0x3f, sizeof distv);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> heap;
    distv[1] = 0;
    heap.push({0, 1});
    while (!heap.empty()) {
        auto [distance, ver] = heap.top(); heap.pop();
        if (st[ver]) continue;
        st[ver] = true;
        for (int i = h[ver]; i != -1; i = ne[i]) {
            int j = e[i];
            if (distv[j] > distance + w[i]) {
                distv[j] = distance + w[i];
                heap.push({distv[j], j});
            }
        }
    }
    return distv[n] == 0x3f3f3f3f ? -1 : distv[n];
}
```

42. Bellman Ford

```
const int N = 510, M = 10010;
int n, m, k, distv[N], backup[N];
struct Edge { int a, b, w; } edges[M];

int bellman_ford() {
    memset(distv, 0x3f, sizeof distv);
    distv[1] = 0;
    for (int i = 0; i < k; i++) {
        memcpy(backup, distv, sizeof distv);
        for (int j = 0; j < m; j++) {
            int a = edges[j].a, b = edges[j].b, w = edges[j].w;
            distv[b] = min(distv[b], backup[a] + w);
        }
    }
    if (distv[n] > 0x3f3f3f3f / 2) return -1;
    return distv[n];
}
```

43. SPFA Shortest Path

```
const int N = 100010, M = 200010;
int n, m, h[N], e[M], w[M], ne[M], idx, distv[N];
bool st[N];

int spfa() {
    memset(distv, 0x3f, sizeof distv);
    queue<int> q;
    distv[1] = 0;
    q.push(1);
    st[1] = true;
    while (!q.empty()) {
        int t = q.front(); q.pop();
        st[t] = false;
        for (int i = h[t]; i != -1; i = ne[i]) {
            int j = e[i];
            if (distv[j] > distv[t] + w[i]) {
                distv[j] = distv[t] + w[i];
            }
        }
    }
}
```

```
        if (!st[j]) q.push(j), st[j] = true;
    }
}
return distv[n] == 0x3f3f3f3f ? -1 : distv[n];
}
```

44. SPFA Negative Cycle

```
const int N = 2010, M = 10010;
int n, m, h[N], e[M], w[M], ne[M], idx, distv[N], cnt[N];
bool st[N];

bool spfa_negative_cycle() {
    queue<int> q;
    for (int i = 1; i <= n; i++) q.push(i), st[i] = true;
    while (!q.empty()) {
        int t = q.front(); q.pop();
        st[t] = false;
        for (int i = h[t]; i != -1; i = ne[i]) {
            int j = e[i];
            if (distv[j] > distv[t] + w[i]) {
                distv[j] = distv[t] + w[i];
                cnt[j] = cnt[t] + 1;
                if (cnt[j] >= n) return true;
                if (!st[j]) q.push(j), st[j] = true;
            }
        }
    }
    return false;
}
```

45. Floyd

```
const int N = 210, INF = 0x3f3f3f3f;
int n, m, k, d[N][N];

void floyd() {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

void init() {
    memset(d, 0x3f, sizeof d);
    for (int i = 1; i <= n; i++) d[i][i] = 0;
}
```

46. Prim Dense MST

```
const int N = 510, INF = 0x3f3f3f3f;
int n, g[N][N], distv[N];
bool st[N];

int prim() {
    memset(distv, 0x3f, sizeof distv);
    int res = 0;
    for (int i = 0; i < n; i++) {
        int t = -1;
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || distv[j] < distv[t])) t = j;
        if (i && distv[t] == INF) return INF;
        if (i) res += distv[t];
        st[t] = true;
        for (int j = 1; j <= n; j++) distv[j] = min(distv[j], g[t][j]);
    }
    return res;
}
```

47. Kruskal MST

```
const int N = 100010, M = 200010;
int n, m, p[N];
struct Edge {
    int a, b, w;
    bool operator<(const Edge& t) const { return w < t.w; }
} edges[M];

int find(int x) {
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal() {
```

```
sort(edges, edges + m);
for (int i = 1; i <= n; i++) p[i] = i;
int res = 0, cnt = 0;
for (int i = 0; i < m; i++) {
    int a = edges[i].a, b = edges[i].b, w = edges[i].w;
    a = find(a), b = find(b);
    if (a != b) {
        p[a] = b;
        res += w;
        cnt++;
    }
}
return cnt < n - 1 ? 0x3f3f3f3f : res;
}
```

48. Bipartite Check By Coloring

```
const int N = 100010, M = 200010;
int n, m, h[N], e[M], ne[M], idx, color[N];

bool dfs(int u, int c) {
    color[u] = c;
    for (int i = h[u]; i != -1; i = ne[i]) {
        int j = e[i];
        if (!color[j]) {
            if (!dfs(j, 3 - c)) return false;
        } else if (color[j] == c) return false;
    }
    return true;
}

bool is_bipartite() {
    for (int i = 1; i <= n; i++)
        if (!color[i] && !dfs(i, 1)) return false;
    return true;
}
```

49. Hungarian Algorithm

```
const int N = 510, M = 100010;
int n1, n2, m, h[N], e[M], ne[M], idx, match[N];
bool st[N];

bool find_match(int x) {
    for (int i = h[x]; i != -1; i = ne[i]) {
        int j = e[i];
        if (!st[j]) {
            st[j] = true;
            if (!match[j] || find_match(match[j])) {
                match[j] = x;
                return true;
            }
        }
    }
    return false;
}

int hungarian() {
    int res = 0;
    for (int i = 1; i <= n1; i++) {
        memset(st, false, sizeof st);
        if (find_match(i)) res++;
    }
    return res;
}
```

50. Trial Division Prime

```
bool is_prime(int x) {
    if (x < 2) return false;
    for (int i = 2; i <= x / i; i++)
        if (x % i == 0) return false;
    return true;
}

vector<pair<int, int>> factorize(int x) {
    vector<pair<int, int>> res;
    for (int i = 2; i <= x / i; i++) {
        if (x % i == 0) {
            int s = 0;
            while (x % i == 0) x /= i, s++;
            res.push_back({i, s});
        }
    }
}
```

```
    if (x > 1) res.push_back({x, 1});
    return res;
}
```

51. Linear Sieve

```
const int N = 1000010;
int primes[N], cnt;
bool st[N];

void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++) {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}
```

52. Divisors

```
vector<int> get_divisors(int x) {
    vector<int> res;
    for (int i = 1; i <= x / i; i++) {
        if (x % i == 0) {
            res.push_back(i);
            if (i != x / i) res.push_back(x / i);
        }
    }
    sort(res.begin(), res.end());
    return res;
}

int gcd_int(int a, int b) {
    return b ? gcd_int(b, a % b) : a;
}

long long lcm_ll(long long a, long long b) {
    return a / std::gcd(a, b) * b;
}
```

53. Divisor Count And Sum Mod

```
const long long MOD = 1000000007;

pair<long long, long long> divisor_count_sum(vector<long long> nums) {
    unordered_map<long long, int> primes;
    for (auto x : nums) {
        for (long long i = 2; i <= x / i; i++) {
            while (x % i == 0) {
                x /= i;
                primes[i]++;
            }
        }
        if (x > 1) primes[x]++;
    }
    long long cnt = 1, sum = 1;
    for (auto [p, a] : primes) {
        cnt = cnt * (a + 1) % MOD;
        long long t = 1;
        while (a--) t = (t * p + 1) % MOD;
        sum = sum * t % MOD;
    }
    return {cnt, sum};
}
```

54. Fast Power Mod

```
long long qmi(long long a, long long k, long long p) {
    long long res = 1 % p;
    while (k) {
        if (k & 1) res = res * a % p;
        a = a * a % p;
        k >>= 1;
    }
    return res;
}

long long mod_inverse_prime(long long a, long long p) {
    return qmi(a, p - 2, p);
}
```


55. Extended GCD And Inverse

```
long long exgcd(long long a, long long b, long long& x, long long& y) {  
    if (!b) {  
        x = 1, y = 0;  
        return a;  
    }  
    long long x1, y1;  
    long long d = exgcd(b, a % b, x1, y1);  
    x = y1;  
    y = x1 - a / b * y1;  
    return d;  
}  
  
long long inv_any_mod(long long a, long long mod) {  
    long long x, y;  
    long long d = exgcd(a, mod, x, y);  
    if (d != 1) return -1;  
    return (x % mod + mod) % mod;  
}
```

56. Linear Congruence

```
bool linear_congruence(long long a, long long b, long long m, long long& x) {  
    long long y;  
    long long d = exgcd(a, m, x, y);  
    if (b % d) return false;  
    x = (__int128)x * (b / d) % m;  
    x = (x % m + m) % m;  
    return true;  
}
```

57. Extended Chinese Remainder

```
long long exCRT(vector<long long> a, vector<long long> m) {  
    long long r1 = a[0], m1 = m[0];  
    for (int i = 1; i < (int)a.size(); i++) {  
        long long r2 = a[i], m2 = m[i];  
        long long x, y;  
        long long d = exgcd(m1, m2, x, y);  
        if ((r2 - r1) % d) return -1;  
        long long t = (__int128)(r2 - r1) / d * x % (m2 / d);  
        r1 = r1 + (__int128)m1 * t;  
        m1 = m1 / d * m2;  
        r1 = (r1 % m1 + m1) % m1;  
    }  
    return r1;  
}
```

58. Gaussian Elimination Double

```
const int N = 110;  
const double eps = 1e-8;  
double a[N][N];  
int n;  
  
int gauss() {  
    int c, r;  
    for (c = 0, r = 0; c < n; c++) {  
        int t = r;  
        for (int i = r; i < n; i++)  
            if (fabs(a[i][c]) > fabs(a[t][c])) t = i;  
        if (fabs(a[t][c]) < eps) continue;  
        for (int i = c; i <= n; i++) swap(a[t][i], a[r][i]);  
        for (int i = n; i >= c; i--) a[r][i] /= a[r][c];  
        for (int i = r + 1; i < n; i++)  
            if (fabs(a[i][c]) > eps)  
                for (int j = n; j >= c; j--) a[i][j] -= a[r][j] * a[i][c];  
        r++;  
    }  
    if (r < n) {  
        for (int i = r; i < n; i++)  
            if (fabs(a[i][n]) > eps) return 2;  
        return 1;  
    }  
    for (int i = n - 1; i >= 0; i--)  
        for (int j = i + 1; j < n; j++) a[i][n] -= a[i][j] * a[j][n];  
    return 0;  
}
```

59. Gaussian Elimination XOR

```
const int N = 110;  
int a[N][N], n;
```

```
int gauss_xor() {
    int r, c;
    for (r = 0, c = 0; c < n; c++) {
        int t = r;
        for (int i = r; i < n; i++)
            if (a[i][c]) { t = i; break; }
        if (!a[t][c]) continue;
        for (int i = c; i <= n; i++) swap(a[t][i], a[r][i]);
        for (int i = r + 1; i < n; i++)
            if (a[i][c])
                for (int j = n; j >= c; j--) a[i][j] ^= a[r][j];
        r++;
    }
    if (r < n) {
        for (int i = r; i < n; i++)
            if (a[i][n]) return 2;
        return 1;
    }
    for (int i = n - 1; i >= 0; i--)
        for (int j = i + 1; j < n; j++) a[i][n] ^= a[i][j] & a[j][n];
    return 0;
}
```

60. Combinations Pascal Mod

```
const int N = 2010, MOD = 1000000007;
int c[N][N];

void init_comb() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j <= i; j++) {
            if (!j) c[i][j] = 1;
            else c[i][j] = (c[i - 1][j] + c[i - 1][j - 1]) % MOD;
        }
    }
}
```

61. Combinations Factorial Mod Prime

```
const int N = 100010, MOD = 1000000007;
long long fact[N], infact[N];

long long qmi(long long a, long long k, long long p) {
    long long res = 1;
    while (k) {
        if (k & 1) res = res * a % p;
        a = a * a % p;
        k >>= 1;
    }
    return res;
}

void init_fact() {
    fact[0] = infact[0] = 1;
    for (int i = 1; i < N; i++) {
        fact[i] = fact[i - 1] * i % MOD;
        infact[i] = infact[i - 1] * qmi(i, MOD - 2, MOD) % MOD;
    }
}

long long C(int a, int b) {
    if (b < 0 || b > a) return 0;
    return fact[a] * infact[b] % MOD * infact[a - b] % MOD;
}
```

62. Lucas Theorem

```
long long qmi(long long a, long long k, long long p) {
    long long res = 1 % p;
    while (k) {
        if (k & 1) res = res * a % p;
        a = a * a % p;
        k >>= 1;
    }
    return res;
}

long long C_mod_p(long long a, long long b, long long p) {
    if (b > a) return 0;
    long long res = 1;
    for (long long i = 1, j = a; i <= b; i++, j--) {
        res = res * j % p;
        res = res * qmi(i, p - 2, p) % p;
    }
}
```

```
    return res;
}

long long lucas(long long a, long long b, long long p) {
    if (a < p && b < p) return C_mod_p(a, b, p);
    return C_mod_p(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}
```

63. Catalan Number Mod Prime

```
long long catalan(int n, long long mod) {
    long long x = 1, y = 1;
    for (int i = 0; i < n; i++) {
        x = x * (2 * n - i) % mod;
        y = y * (i + 1) % mod;
    }
    long long c = x * qmi(y, mod - 2, mod) % mod;
    return c * qmi(n + 1, mod - 2, mod) % mod;
}
```

64. Nim And SG

```
bool nim_win(vector<int> a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x != 0;
}

const int N = 10010, M = 110;
int s[M], f[N];

int sg(int x) {
    if (f[x] != -1) return f[x];
    unordered_set<int> S;
    for (int i = 0; s[i] <= x; i++) S.insert(sg(x - s[i]));
    for (int i = 0;; i++)
        if (!S.count(i)) return f[x] = i;
}
```

65. 0/1 Knapsack

```
const int N = 1010;
int n, m, f[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        int v, w; cin >> v >> w;
        for (int j = m; j >= v; j--) f[j] = max(f[j], f[j - v] + w);
    }
    cout << f[m] << '\n';
    return 0;
}
```

66. Complete Knapsack

```
const int N = 1010;
int n, m, f[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        int v, w; cin >> v >> w;
        for (int j = v; j <= m; j++) f[j] = max(f[j], f[j - v] + w);
    }
    cout << f[m] << '\n';
    return 0;
}
```

67. Multiple Knapsack Binary Optimization

```
const int N = 25000;
int n, m, f[N];
struct Good { int v, w; } goods[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
```

```
int v, w, s; cin >> v >> w >> s;
for (int k = 1; k <= s; k <= 1) {
    cnt++;
    goods[cnt] = {v * k, w * k};
    s -= k;
}
if (s) goods[++cnt] = {v * s, w * s};
}
for (int i = 1; i <= cnt; i++)
    for (int j = m; j >= goods[i].v; j--)
        f[j] = max(f[j], f[j - goods[i].v] + goods[i].w);
cout << f[m] << '\n';
return 0;
}
```

68. Group Knapsack

```
const int N = 110;
int n, m, s[N], v[N][N], w[N][N], f[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> s[i];
        for (int j = 0; j < s[i]; j++) cin >> v[i][j] >> w[i][j];
    }
    for (int i = 1; i <= n; i++)
        for (int j = m; j >= 0; j--)
            for (int k = 0; k < s[i]; k++)
                if (j >= v[i][k]) f[j] = max(f[j], f[j - v[i][k]] + w[i][k]);
    cout << f[m] << '\n';
    return 0;
}
```

69. Longest Increasing Subsequence

```
int lis_nlogn(vector<int> a) {
    vector<int> q;
    for (int x : a) {
        auto it = lower_bound(q.begin(), q.end(), x);
        if (it == q.end()) q.push_back(x);
        else *it = x;
    }
    return q.size();
}

int lis_n2(vector<int> a) {
    int n = a.size(), res = 0;
    vector<int> f(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) f[i] = max(f[i], f[j] + 1);
        res = max(res, f[i]);
    }
    return res;
}
```

70. Longest Common Subsequence

```
const int N = 1010;
char a[N], b[N];
int f[N][N];

int lcs(int n, int m) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            f[i][j] = max(f[i - 1][j], f[i][j - 1]);
            if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i - 1][j - 1] + 1);
        }
    }
    return f[n][m];
}
```

71. Edit Distance

```
const int N = 1010;
char a[N], b[N];
int f[N][N];

int edit_distance(int n, int m) {
    for (int i = 0; i <= n; i++) f[i][0] = i;
    for (int j = 0; j <= m; j++) f[0][j] = j;
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= m; j++) {
            f[i][j] = min(f[i - 1][j], f[i][j - 1]) + 1;
            f[i][j] = min(f[i][j], f[i - 1][j - 1] + (a[i] != b[j]));
        }
    }
    return f[n][m];
}
```

72. Interval DP: Stone Merge

```
const int N = 310;
int n, s[N], f[N][N];

int interval_dp() {
    memset(f, 0x3f, sizeof f);
    for (int len = 1; len <= n; len++) {
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            if (l == r) f[l][r] = 0;
            else {
                for (int k = l; k < r; k++)
                    f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r] + s[r] - s[l - 1]);
            }
        }
    }
    return f[1][n];
}
```

73. Integer Partition Count

```
const int N = 1010, MOD = 1000000007;
int f[N];

int partition_count(int n) {
    f[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = i; j <= n; j++)
            f[j] = (f[j] + f[j - i]) % MOD;
    return f[n];
}
```

74. Digit DP Skeleton

```
using ll = long long;
vector<int> num;
ll memo[20][20][2];
bool vis[20][20][2];

ll dfs_digit(int pos, int pre, bool started, bool tight) {
    if (pos == -1) return started;
    if (!tight && vis[pos][pre][started]) return memo[pos][pre][started];
    int up = tight ? num[pos] : 9;
    ll res = 0;
    for (int d = 0; d <= up; d++) {
        bool ns = started || d != 0;
        res += dfs_digit(pos - 1, ns ? d : 0, ns, tight && d == up);
    }
    if (!tight) {
        vis[pos][pre][started] = true;
        memo[pos][pre][started] = res;
    }
    return res;
}

ll solve_digit(ll x) {
    num.clear();
    while (x) num.push_back(x % 10), x /= 10;
    if (num.empty()) num.push_back(0);
    memset(vis, 0, sizeof vis);
    return dfs_digit((int)num.size() - 1, 0, false, true);
}
```

75. State Compression DP: Hamilton Path

```
const int N = 20;
int n, w[N][N], f[1 << N][N];

int hamilton() {
    memset(f, 0x3f, sizeof f);
    f[1][0] = 0;
    for (int i = 0; i < (1 << n); i++) {
        for (int j = 0; j < n; j++) {
            if (i >> j & 1) {
                for (int k = 0; k < n; k++) {
                    if ((i - (1 << j)) >> k & 1) {
```

```
        f[i][j] = min(f[i][j], f[i - (1 << j)][k] + w[k][j]);
    }
}
}
}
return f[(1 << n) - 1][n - 1];
}
```

76. State Compression DP: Mondrian Dream

```
const int N = 12, M = 1 << N;
long long f[N][M];
bool st[M];
vector<int> state[M];

long long mondrian(int n, int m) {
    for (int i = 0; i < 1 << n; i++) {
        st[i] = true;
        int cnt = 0;
        for (int j = 0; j < n; j++) {
            if (i >> j & 1) {
                if (cnt & 1) st[i] = false;
                cnt = 0;
            } else cnt++;
        }
        if (cnt & 1) st[i] = false;
    }
    for (int i = 0; i < 1 << n; i++) {
        state[i].clear();
        for (int j = 0; j < 1 << n; j++)
            if ((i & j) == 0 && st[i | j]) state[i].push_back(j);
    }
    memset(f, 0, sizeof f);
    f[0][0] = 1;
    for (int i = 1; i <= m; i++)
        for (int j = 0; j < 1 << n; j++)
            for (int k : state[j]) f[i][j] += f[i - 1][k];
    return f[m][0];
}
```

77. Tree DP: No Boss Party

```
const int N = 6010;
int n, happy[N], h[N], e[N], ne[N], idx, f[N][2];
bool has_father[N];

void add(int a, int b) {
    e[idx] = b; ne[idx] = h[a]; h[a] = idx++;
}

void dfs(int u) {
    f[u][1] = happy[u];
    for (int i = h[u]; i != -1; i = ne[i]) {
        int j = e[i];
        dfs(j);
        f[u][0] += max(f[j][0], f[j][1]);
        f[u][1] += f[j][0];
    }
}
```

78. Memoized Search: Skiing

```
const int N = 310;
int n, m, hgt[N][N], f[N][N];
int dx[4] = {-1, 0, 1, 0};
int dy[4] = {0, 1, 0, -1};

int dp(int x, int y) {
    int& v = f[x][y];
    if (v != -1) return v;
    v = 1;
    for (int i = 0; i < 4; i++) {
        int a = x + dx[i], b = y + dy[i];
        if (a >= 1 && a <= n && b >= 1 && b <= m && hgt[a][b] < hgt[x][y])
            v = max(v, dp(a, b) + 1);
    }
    return v;
}
```

79. Greedy: Interval Selection

```
int interval_selection(vector<pair<int, int>> segs) {
    sort(segs.begin(), segs.end(), [](auto& a, auto& b) {
        return a.second < b.second;
    });
}
```

```
});  
int res = 0, ed = -2e9;  
for (auto [l, r] : segs) {  
    if (l > ed) {  
        res++;  
        ed = r;  
    }  
}  
return res;  
}
```

80. Greedy: Interval Grouping

```
int interval_grouping(vector<pair<int,int>> segs) {  
    sort(segs.begin(), segs.end());  
    priority_queue<int, vector<int>, greater<int>> heap;  
    for (auto [l, r] : segs) {  
        if (heap.empty() || heap.top() >= l) heap.push(r);  
        else {  
            heap.pop();  
            heap.push(r);  
        }  
    }  
    return heap.size();  
}
```

81. Greedy: Interval Cover

```
int interval_cover(vector<pair<int,int>> segs, int st, int ed) {  
    sort(segs.begin(), segs.end());  
    int res = 0;  
    for (int i = 0; i < (int)segs.size(); ) {  
        int j = i, r = -2e9;  
        while (j < (int)segs.size() && segs[j].first <= st) {  
            r = max(r, segs[j].second);  
            j++;  
        }  
        if (r < st) return -1;  
        res++;  
        if (r >= ed) return res;  
        st = r;  
        i = j;  
    }  
    return -1;  
}
```

82. Greedy: Huffman Merge

```
long long huffman_cost(vector<int> a) {  
    priority_queue<int, vector<int>, greater<int>> heap;  
    for (int x : a) heap.push(x);  
    long long res = 0;  
    while (heap.size() > 1) {  
        int x = heap.top(); heap.pop();  
        int y = heap.top(); heap.pop();  
        res += x + y;  
        heap.push(x + y);  
    }  
    return res;  
}
```

83. Greedy: Median Absolute Inequality

```
long long min_abs_sum(vector<int> a) {  
    sort(a.begin(), a.end());  
    int median = a[a.size() / 2];  
    long long res = 0;  
    for (int x : a) res += abs(x - median);  
    return res;  
}
```

84. Greedy: Sorting Inequality

```
long long min_waiting_sum(vector<int> t) {  
    sort(t.begin(), t.end());  
    long long res = 0, prefix = 0;  
    for (int i = 0; i < (int)t.size(); i++) {  
        res += prefix;  
        prefix += t[i];  
    }  
    return res;  
}
```

85. Fast IO Scanner

```
struct FastScanner {
    static const int S = 1 << 20;
    int idx = 0, size = 0;
    char buf[S];
    char getch() {
        if (idx >= size) {
            size = fread(buf, 1, S, stdin);
            idx = 0;
            if (size == 0) return 0;
        }
        return buf[idx++];
    }
    template <class T>
    bool read(T& x) {
        char c; T sign = 1; x = 0;
        c = getch();
        if (!c) return false;
        while (c != '-' && (c < '0' || c > '9')) {
            c = getch();
            if (!c) return false;
        }
        if (c == '-') sign = -1, c = getch();
        while (c >= '0' && c <= '9') {
            x = x * 10 + c - '0';
            c = getch();
        }
        x *= sign;
        return true;
    }
};
```

86. Output Helpers

```
template <class T>
void print_vector(const vector<T>& a, char sep = ' ', char end = '\n') {
    for (int i = 0; i < (int)a.size(); i++) {
        if (i) cout << sep;
        cout << a[i];
    }
    cout << end;
}

template <class T>
bool chmax(T& a, const T& b) {
    if (a < b) { a = b; return true; }
    return false;
}

template <class T>
bool chmin(T& a, const T& b) {
    if (a > b) { a = b; return true; }
    return false;
}
```